

第39章 DAC—输出正弦波

本章参考资料：《STM32F4XX 文参考手册》DAC 章节。

学习本章时，配合《STM32F4XX 文参考手册》DAC 章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

39.1 DAC 简介

DAC 为数字/模拟转换模块，故名思议，它的作用就是把输入的数字编码，转换成对应的模拟电压输出，它的功能与 ADC 相反。在常见的数字信号系统中，大部分传感器信号被化成电压信号，而 ADC 把电压模拟信号转换成易于计算机存储、处理的数字编码，由计算机处理完成后，再由 DAC 输出电压模拟信号，该电压模拟信号常常用来驱动某些执行器件，使人类易于感知。如音频信号的采集及还原就是这样一个过程。

STM32 具有片上 DAC 外设，它的分辨率可配置为 8 位或 12 位的数字输入信号，具有两个 DAC 输出通道，这两个通道互不影响，每个通道都可以使用 DMA 功能，都具有出错检测能力，可外部触发。

39.2 DAC 功能框图剖析

STM32 的 DAC 模块框图见图 39-1。

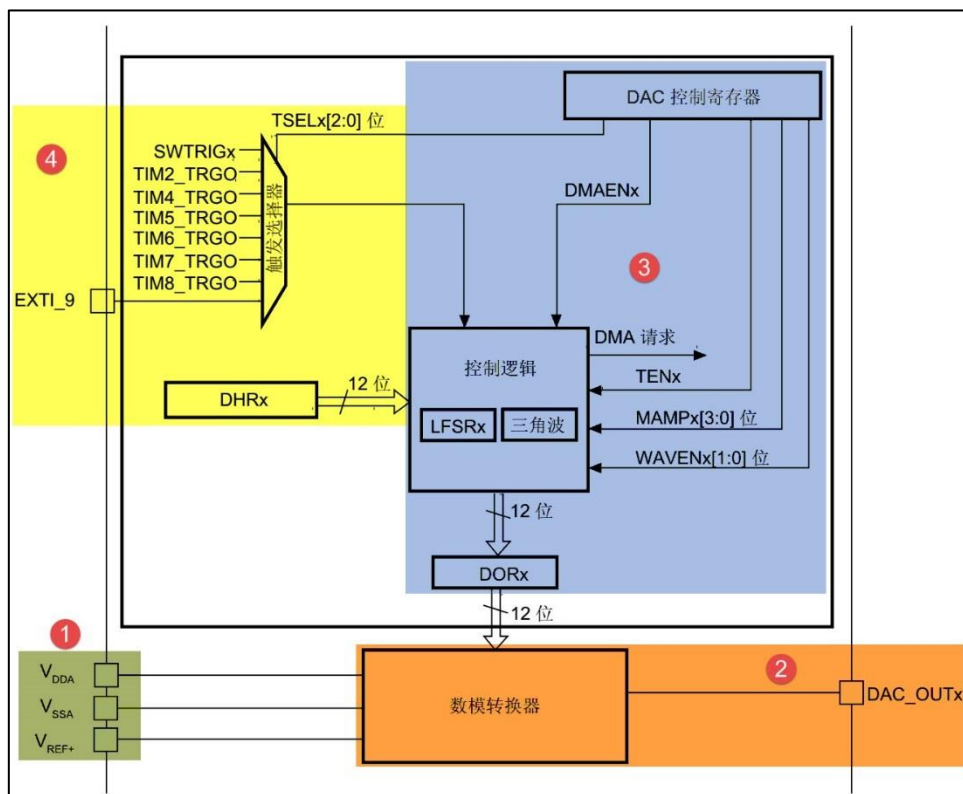


图 39-1 DAC 功能框图

整个 DAC 模块围绕框图下方的“数字至模拟转换器 x”展开，它的左边分别是参考电源的引脚： V_{DDA} 、 V_{SSA} 及 V_{ref+} ，其中 STM32 的 DAC 规定了它的参考电压 V_{ref+} 输入范围为 2.4——3.3V。“数字至模拟转换器 x”的输入为 DAC 的数据寄存器“DORx”的数字编码，经过它转换得的模拟信号由图中右侧的“DAC_OUTx”输出。而数据寄存器“DORx”又受“控制逻辑”支配，它可以控制数据寄存器加入一些伪噪声信号或配置产生三角波信号。图中的左上角为 DAC 的触发源，DAC 根据触发源的信号来进行 DAC 转换，其作用就相当于 DAC 转换器的开关，它可以配置的触发源为外部中断源触发、定时器触发或软件控制触发。如本章实验中需要控制正弦波的频率，就需要定时器定时触发 DAC 进行数据转换。

1. 参考电压

与 ADC 外设类似，DAC 也使用 V_{REF+} 引脚作为参考电压，在设计原理图的时候一般把 V_{SSA} 接地，把 V_{REF+} 和 V_{DDA} 接 3.3V，可得到 DAC 的输出电压范围为：0~3.3V。

如果能让输出的电压范围变宽，可以在外部加一个电压调理电路，把 0~3.3V 的 DAC 输出抬升到特定的范围即可。

2. 数模转换及输出通道

框图中的“数字至模拟转换器 x”是核心部件，整个 DAC 外设都围绕它而展开。它以左边的 V_{REF+} 作为参考电源，以 DAC 的数据寄存器“DORx”的数字编码作为输入，经过它转换得的模拟信号由右侧的“DAC_OUTx”通道输出。其中各个部件中的“x”是指设备的标号，在 STM32 中具有 2 个这样的 DAC 部件，每个 DAC 有 1 个对应的输出通道连接到特定的引脚，即：PA4-通道 1，PA5-通道 2，为避免干扰，使用 DAC 功能时，DAC 通道引脚需要被配置成模拟输入功能（AIN）。

3. 触发源及 DHRx 寄存器

在使用 DAC 时，不能直接对上述 DORx 寄存器写入数据，任何输出到 DAC 通道 x 的数据都必须写入到 DHRx 寄存器中（其中包含 DHR8Rx、DHR12Lx 等，根据数据对齐方向和分辨率的情况写入到对应的寄存器中）。

数据被写入到 DHRx 寄存器后，DAC 会根据触发配置进行处理，若使用硬件触发，则 DHRx 中的数据会在 3 个 APB1 时钟周期后传输至 DORx，DORx 随之输出相应的模拟电压到输出通道；若 DAC 设置为外部事件触发，可以使用定时器（TIMx_TRGO）、EXTI_9 信号或软件触发（SWTRIGx）这几种方式控制数据 DAC 转换的时机，例如使用定时器触发，配合不同时刻的 DHRx 数据，可实现 DAC 输出正弦波的功能。

39.3 DAC 初始化结构体详解

在 ST 的标准库中，把控制 DAC 相关的各种配置封装到了结构体 DAC_InitTypeDef 中，它主要包含了 DAC_CR 控制寄存器的各寄存器位的配置，见代码清单 39-1。

代码清单 39-1 DAC_InitTypeDef 结构体

```
1 typedef struct {
2     /*DAC 触发方式 */
3     uint32_t DAC_Trigger;
4
5     /*是否自动输出噪声或三角波 */
6     uint32_t DAC_WaveGeneration;
7
8     /*选择噪声生成器的低通滤波或三角波的幅值 */
9     uint32_t DAC_LFSRUnmask_TriangleAmplitude;
10
11     /*选择是否使能输出缓冲器 */
12     uint32_t DAC_OutputBuffer;
13
14 } DAC_InitTypeDef;
```

各个结构体成员的介绍如下，解说中各模式后括号内的英文为该模式在标准库中使用宏：

(1) DAC_Trigger

本成员用于配置 DAC 的触发模式，当 DAC 产生相应的触发事件时，才会把 DHRx 寄存器的值转移到 DORx 寄存器中进行转换。本结构体成员可以选择的触发模式如下：硬件触发模式（DAC_Trigger_None），DHRx 寄存器内的数据会在 3 个 APB1 时钟周期内自动转换至 DORx 进行转换；定时器触发模式

（DAC_Trigger_T2/4/5/6/7/8_TRGO），使用定时器 2、4、5、6、7、8 控制 DHRx 寄存器的数据按时间转移到 DORx 中进行转换，利用这种方式可以输出特定的波形；EXTI_9 触发方式（DAC_Trigger_Ext_IT9），当产生 EXTI_9 事件时（如 GPIO 中断事件），触发转换；软件触发模式（DAC_Trigger_Software），在本模式下，向 DAC_SWTRIGR 寄存器写入配置即可触发信号进行转换。

(2) DAC_WaveGeneration

本成员用于设置是否使用 DAC 输出伪噪声或三角波

（DAC_WaveGeneration_None/Noise/Triangle），使用伪噪声和三角波输出时，DAC 都会把 LFSR 寄存器的值叠加到 DHRx 数值上，产生伪噪声和三角波，若希望产生自定义的输出时，直接配置为 DAC_WaveGeneration_None 即可。

(3) DAC_LFSRUnmask_TriangleAmplitude

本成员通过控制 DAC_CR 的 MAMP2 位设置 LFSR 寄存器位的数据，即当使用伪噪声或三角波输出时要叠加到 DHRx 的值，非噪声或三角波输出模式下，本配置无效。使用伪噪声输出时 LFSR=0xAAA，MAMP2 寄存器位可以屏蔽 LFSR 的某些位，这时把本结构体成员赋值为 DAC_LFSRUnmask_Bit0~DAC_LFSRUnmask_Bit11_0 等宏即可；使用三角波输出时，本结构体设置三角波的最大幅值，可选择为

DAC_TriangleAmplitude_1~DAC_TriangleAmplitude_4096 等宏，见图 39-2，DAC 在 DHRx 值的基础上升，幅值达到 MAMPx 设置的最大幅度时下降，形成三角波的输出。

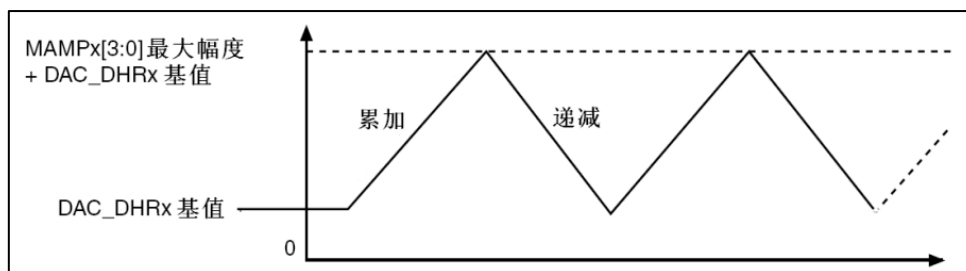


图 39-2 DAC 输出三角波

(4) DAC_OutputBuffer

本结构体成员用于控制是否使能 DAC 的输出缓冲

（DAC_OutputBuffer_Enable/Disable），使能了 DAC 的输出缓冲后可以减小输出阻抗，适合直接驱动一些外部负载。

39.4 DAC 输出正弦波实验

利用 STM32 的 DAC 配合 TIM 定时器，可以输出随时间变化的电压，本章的实验以输出正弦波为例，演示如何控制输出电压波形。

39.4.1 硬件设计

STM32 的 DAC 外设有固定的输出通道，分别为 PA4 和 PA5，不过，霸天虎开发板已经在摄像头接口和电容按键使用了这两个引脚，所以用作 DAC 通道输出电压时会受到干扰，做本实验时需要注意不要给开发板接入摄像头，并且把电容按键旁边的跳线帽拔掉，见图 31-5。

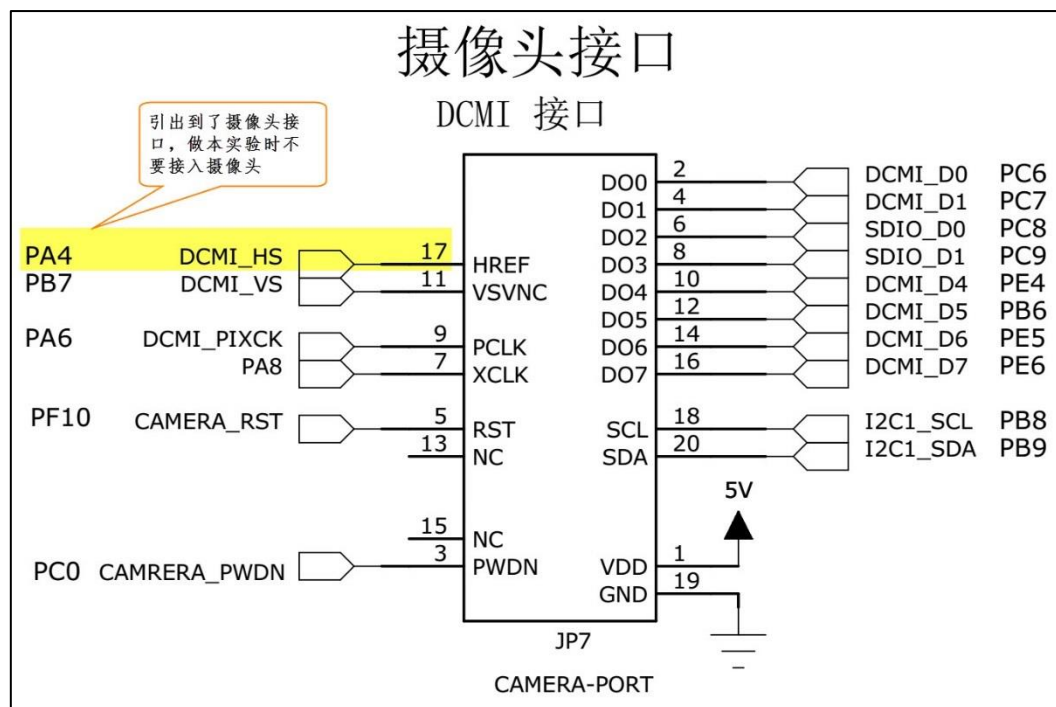


图 39-3 DCMI 摄像头接口占用了 DAC 使用的输出通道

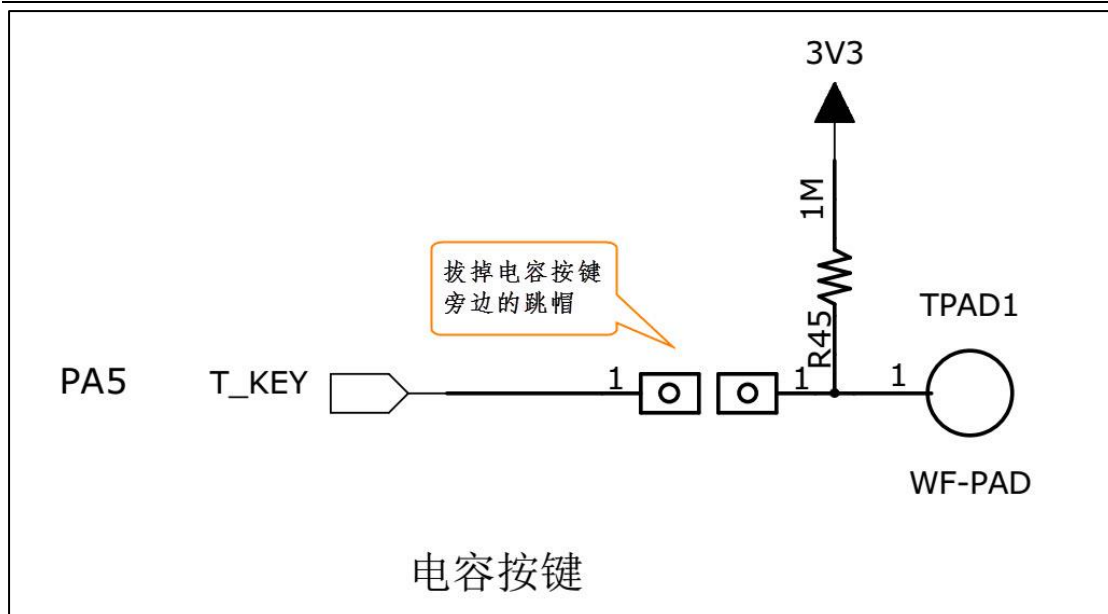


图 39-4 电容按键占用了 DAC 使用的输出通道

在设计 DAC 专门的实际应用时，DAC 的输出通道应独占，不与其它设备共用。本开发板的设计是考虑到各种资源分配，才不得不占用 DAC 通道的。

注意前面的引脚占用问题后，在实验时直接使用示波器测量 PA4 和 PA5 引脚的输出即可。

39.4.2 软件设计

为了使工程更加有条理，我们把 DAC 控制相关的代码独立分开存储，方便以后移植。新建“bsp_dac.c”及“bsp_dac.h”文件，这些文件也可根据您的喜好命名，它们不属于 STM32 标准库的内容，是由我们自己根据应用需要编写的。

1. 编程要点

- (1) 计算获取正弦波数据表；
- (2) 根据正弦波数据表的周期内点数和周期计算定时器触发间隔；
- (3) 初始化 DAC 输出通道，初始化 DAC 工作模式；
- (4) 配置触发 DAC 用的定时器；
- (5) 配置 DMA 自动转运正弦波数据表。

配置完成后，即可在 PA4、PA5 引脚中检测到信号输出。

2. 代码分析

生成正弦波数据表

要输出正弦波，实质是要控制 DAC 以 $v=\sin(t)$ 的正弦函数关系输出电压，其中 v 为电压输出， t 为时间。

而由于模拟信号连续而数字信号是离散的，所以使用 DAC 产生正弦波时，只能按一定时间间隔输出正弦曲线上的点，在该时间段内输出相同的电压值，若缩短时间间隔，提高单个周期内的输出点数，可以得到逼近连续正弦波的图形，见图 39-5，若在外电路加上适当的电容滤波，可得到更完美的图形。

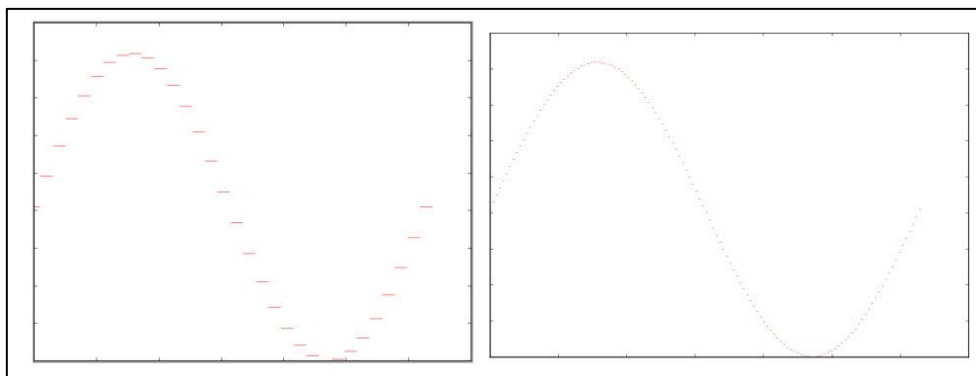


图 39-5 DAC 按点输出正弦波数据(左：32 个点，右：128 个点)

由于正弦曲线是周期函数，所以只需要得到单个周期内的数据后按周期重复即可，而单个周期内取样输出的点数又是有限的，所以为了得到呈 $v=\sin(t)$ 函数关系电压值的数据通常不会实时计算获取，而是预先计算好函数单个周期内的电压数据表，并且转化成以 DAC 寄存器表示的值。

如 $\sin$ 函数值的范围为 $[-1: +1]$ ，而 STM32 的 DAC 输出电压范围为 $[0\sim 3.3]\text{V}$ ，按 12 位 DAC 分辨率表示的方法，可写入寄存器的最大值为 $2^{12} = 4096$ ，即范围为 $[0:4096]$ 。所以，实际输出时，会进行如下处理：

- (1) 抬升 $\sin$ 函数的输出为正值： $v = \sin(t) + 1$ ，此时， v 的输出范围为 $[0:2]$ ；
- (2) 扩展输出至 DAC 的全电压范围： $v = 3.3 * (\sin(t) + 1) / 2$ ，此时， v 的输出范围为 $[0:3.3]$ ，正是 DAC 的电压输出范围，扩展至全电压范围可以充分利用 DAC 的分辨率；
- (3) 把电压值以 DAC 寄存器的形式表示： $\text{Reg_val} = 2^{12} / 3.3 * v = 2^{11} * (\sin(t) + 1)$ ，此时，存储到 DAC 寄存器的值范围为 $[0:4096]$ ；
- (4) 实践证明，在 $\sin(t)$ 的单个周期内，取 32 个点进行电压输出已经能较好地还原正弦波形，所以在 $t \in [0:2\pi]$ 区间内等间距根据上述 Reg_val 公式运算得到 32 个寄存器值，即可得到正弦波表；
- (5) 控制 DAC 输出时，每隔一段相同的时间从上述正弦波表中取出一个新数据进行输出，即可输出正弦波。改变间隔时间的单位长度，可以改变正弦波曲线的周期。

为方便起见，我们使用了 Python 和 Matlab 脚本制作正弦波表，脚本的代码存储在本工程的目录下，感兴趣可以打开文件查看，以下列出 Python 脚本代码，见代码清单 39-2。代码清单 39-2 制作正弦波数据表的 python 脚本（工程目录下的 sinWave.py 文件）

```
1  #! python3
2  #coding=utf-8
3
4  """
5  Python 版本: 3.x
6  外部库: matplotlib1.5.3、numpy1.11.2
7
8  运行方式:
9  在命令行中输入: python sinWave.py
10
11  运行结果:
12  命令行中会打印计算得的各点数据,
13  在当前目录下会生成 py_dac_sinWav.c 文件, 包含上述数据,
14  并且会弹出描绘曲线的对话框。
15  """
16
17  import matplotlib.pyplot as plt
18  import numpy as np
19  import math
20
21  #修改本变量可以更改点数, 如 16、32、64 等
22  POINT_NUM = 32
23
24  pi = math.pi
25
26  #一个周期 POINT_NUM 个点
27  n = np.linspace(0,2*pi,POINT_NUM)
28
29  #计算 POINT_NUM 个点的正弦值
30  a = map(math.sin,n)
31
32  r = []
33  for i in a:
34      #调整幅值至在 0~1 区间
35      i+=1
36
37      #按 3.3V 电压调整幅值
38      i*= 3.3/2
39
40      #求取 dac 数值, 12 位 dac LSB = 3.3V/2**12
41      ri = round(i*2**12/3.3)
42
43      #检查参数
44      if ri >= 4095:
45          ri = 4095
46
47      #得到 dac 数值序列
48      r.append( ri )
49
50
51  print(list(map(int,r)))
52
53  #写入序列到文件
54  with open("py_dac_sinWav.c",'w',encoding= 'gb2312') as f:
55      print(list(map(int,r)),file= f)
56
57  #绘图
58  plt.plot(n,r,"-o")
59  plt.show()
60
```

Python 脚本的实现原理就是前面介绍的正弦波数据表的制作过程, 运行后, 该脚本把得到的正弦波表数据输出到目录下的 py_dac_sinWav.c 文件中, 见代码清单 39-3, 并且根

据取样点描绘出示意图，见图 39-6。Matlab 脚本原理相同，此处不再列出，实际上使用 C 语言也能制作正弦波表，只是画图不方便而已。

代码清单 39-3 生成的正弦波数据表

[2048, 2460, 2856, 3218, 3532, 3786, 3969, 4072, 4093, 4031, 3887, 3668, 3382, 3042, 2661, 2255, 1841, 1435, 1054, 714, 428, 209, 65, 3, 24, 127, 310, 564, 878, 1240, 1636, 2048]

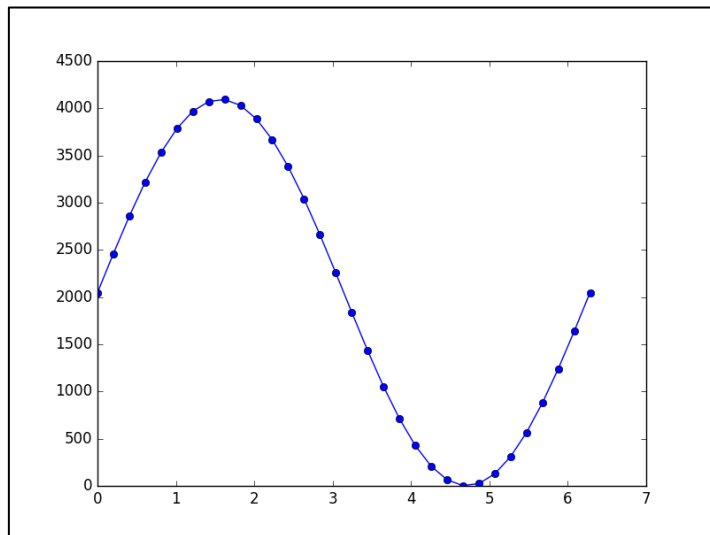


图 39-6 python 脚本根据正弦波表描绘的曲线图

DAC 宏定义

制作好正弦波数据表后，开始使用 MDK 编写 STM32 的 DAC 工程，首先设置好相关的宏，见代码清单 39-4。

代码清单 39-4 DAC 宏定义（bsp_dac.h 文件）

```
1 //DAC 通道数据输出寄存器地址 0x40007420
2 #define DAC_DHR12RD_Address (uint32_t) (DAC_BASE+0x20)
3
4 #define DAC_CLK RCC_APB1Periph_DAC
5 #define DAC_TIM TIM2
6 #define DAC_TIM_CLK RCC_APB1Periph_TIM2
7 #define DAC_TRIGGER DAC_Trigger_T2_TRGO
8
9 #define DAC_DMA_CLK RCC_AHB1Periph_DMA1
10 #define DAC_CHANNEL DMA_Channel_7
11 #define DAC_DMA_STREAM DMA1_Stream5
12
13 #define DAC_CH1_GPIO_CLK RCC_AHB1Periph_GPIOA
14 #define DAC_CH1_GPIO_PORT GPIOA
15 #define DAC_CH1_GPIO_PIN GPIO_Pin_4
16 #define DAC_CH1_CHANNEL DAC_Channel_1
17
18 #define DAC_CH2_GPIO_CLK RCC_AHB1Periph_GPIOA
19 #define DAC_CH2_GPIO_PORT GPIOA
20 #define DAC_CH2_GPIO_PIN GPIO_Pin_5
21 #define DAC_CH2_CHANNEL DAC_Channel_2
```

代码开头部分定义的宏 DAC_DHR12RD_ADDRESS 是寄存器 DHR12RD 的地址，该寄存器是 12 位右对齐的双通道寄存器，见图 39-7。在本实验中将会使用 DMA 把正弦波数据的点数据赋值到该寄存器中，往该寄存器赋值后的数据会在 DAC 被触发的时候搬运到 2 个 DAC 转换器，然后在这 2 个通道中输出以 12 位右对齐表示的这两个通道的电压。DAC 中还有其它寄存器，它们的功能类似，可以在《STM32 中文参考手册》中了解到。

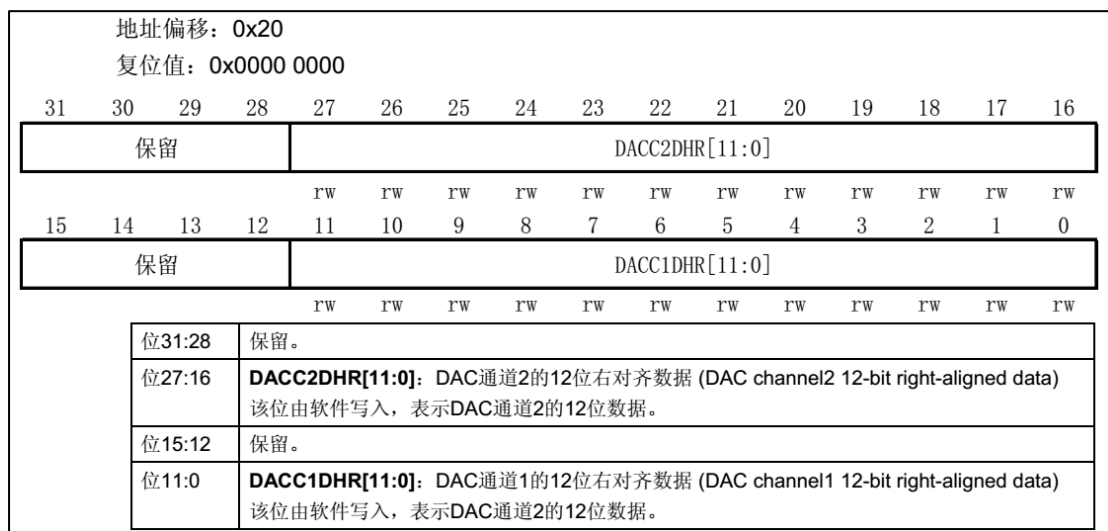


图 39-7 DHR12RD 寄存器说明

其余的代码定义了 DAC 输出通道相关的引脚时钟、引脚号等内容。

DAC GPIO 和模式配置

代码清单 39-5 DAC GPIO 和模式配置

```

1 /**
2  * @brief  使能 DAC 的时钟，初始化 GPIO
3  * @param  无
4  * @retval 无
5  */
6 static void DAC_Config(void)
7 {
8     GPIO_InitTypeDef GPIO_InitStructure;
9     DAC_InitTypeDef  DAC_InitStructure;
10
11     /* 使能 GPIOA 时钟 */
12     RCC_AHB1PeriphClockCmd(DAC_CH1_GPIO_CLK|DAC_CH2_GPIO_CLK, ENABLE);
13
14     /* 使能 DAC 时钟 */
15     RCC_APB1PeriphClockCmd(DAC_CLK, ENABLE);
16
17     /* DAC 的 GPIO 配置，模拟输入 */
18     GPIO_InitStructure.GPIO_Pin = DAC_CH1_GPIO_PIN;
19     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
20     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
21     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
22     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
23     GPIO_Init(DAC_CH1_GPIO_PORT, &GPIO_InitStructure);
24
25     GPIO_InitStructure.GPIO_Pin = DAC_CH2_GPIO_PIN;
26     GPIO_Init(DAC_CH2_GPIO_PORT, &GPIO_InitStructure);
27

```

```
28  /* 配置 DAC 通道 1 */
29  DAC_InitStructure.DAC_Trigger = DAC_TRIGGER; //使用 TIM2 作为触发源
30  DAC_InitStructure.DAC_WaveGeneration = DAC_WaveGeneration_None; //不使用波形发生器
31  DAC_InitStructure.DAC_OutputBuffer = DAC_OutputBuffer_Enable; //不使用 DAC 输出缓冲
32  //三角波振幅（本实验没有用到，可配置成任意值，但本结构体成员不能为空）
33  DAC_InitStructure.DAC_LFSRUnmask_TriangleAmplitude = DAC_TriangleAmplitude_4095;
34  DAC_Init(DAC_CH1_CHANNEL, &DAC_InitStructure);
35
36  /* 配置 DAC 通道 2 */
37  DAC_Init(DAC_CH2_CHANNEL, &DAC_InitStructure);
38
39  /* 配置 DAC 通道 1、2 */
40  DAC_Cmd(DAC_Channel_1, ENABLE);
41  DAC_Cmd(DAC_Channel_2, ENABLE);
42
43  /* 使能 DAC 的 DMA 请求 */
44  DAC_DMAMCmd(DAC_Channel_1, ENABLE);
45
46 }
```

在 DAC_Config 函数中，完成了 DAC 通道的 GPIO 的初始化和 DAC 模式配置。其中 GPIO 按照要求被配置为模拟输入模式（没有模拟输出模式），在该模式下才能正常输出模拟信号。

配置 DAC 工作模式时，则使用了 DAC_InitTypeDef 类型的初始化结构体，把 DAC 通道 1 和 2 都配置成了使用定时器 TIM2 触发、不使用波形发生器以及不使用 DAC 输出缓冲的模式。

初始化完 GPIO 和 DAC 模式后，还使用了 DAC_Cmd、DAC_DMAMCmd 函数使能了通道以及 DMA 的请求。由于本实验中对 DAC1 和 2 的操作是同步的，所以只要把 DMA 与 DAC 通道 2 关联起来即可，当使用 DMA 设置通道 2 的数据值时，同时更新通道 1 的内容。

定时器配置及计算正弦波的频率

初始化完 DAC 后，需要配置触发用的定时器，设定每次触发的间隔，以达到控制正弦波周期的目的。

代码清单 39-6 配置定时器

```
1
2 /**
3  * @brief 配置 TIM
4  * @param 无
5  * @retval 无
6  */
7 static void DAC_TIM_Config(void)
8 {
9
10     TIM_TimeBaseInitTypeDef    TIM_TimeBaseStructure;
11
12     /* 使能 TIM2 时钟，TIM2CLK 为 168M */
13     RCC_APB1PeriphClockCmd(DAC_TIM_CLK, ENABLE);
14
15     /* TIM2 基本定时器配置 */
16     TIM_TimeBaseStructure.TIM_Period = 19; //定时周期 20
17     TIM_TimeBaseStructure.TIM_Prescaler = 0x0; //预分频，不分频 168M / (0+1) = 168M
18     TIM_TimeBaseStructure.TIM_ClockDivision = 0x0; //时钟分频系数
19     TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //向上计数模式
20     TIM_TimeBaseInit(DAC_TIM, &TIM_TimeBaseStructure);
21 }
```

```

22  /* 配置 TIM2 触发源 */
23  TIM_SelectOutputTrigger(DAC_TIM, TIM_TRGOSource_Update);
24
25  /* 使能 TIM2 */
26  TIM_Cmd(DAC_TIM, ENABLE);
27
28 }

```

因为前面的 DAC 配置了 TIM2 当触发源，所以这里将对 TIM2 进行配置。TIM2 的定时周期被配置为 19，向上计数，不分频。即 TIM2 每隔 $(19+1) \times (1/168M)$ 秒就会触发一次 DAC 事件，作 DAC 触发源使用的定时器并不需要设置中断，当定时器计数器向上计数至指定的值时，产生 Update 事件，同时触发 DAC 把 DHRx 寄存器的数据转移到 DORx，从而开始进行转换。

根据定时器的配置，可推算出正弦波频率的计算方式：

按默认配置，STM32 系统时钟周期为：

$$T_{systick} = 1/168000000,$$

定时器 TIM2 的单个时钟周期：

$$T_{tim} = (TIM_Prescaler + 1) \times T_{systick},$$

定时器触发周期：

$$T_{update} = (TIM_Period + 1) \times T_{tim},$$

根据正弦波单个周期的点数 N，求出正弦波单个周期时间为：

$$T_{sin} = T_{update} \times N,$$

对应正弦波的频率为：

$$f_{sin} = \frac{1}{T_{sin}} = \frac{1}{T_{systick} \times (TIM_Prescaler + 1) \times (TIM_Period + 1) \times N}$$

根据上述公式，代入本工程的配置，可得本实验中的正弦波频率为 262500：

$$f_{sin} = \frac{1}{T_{sin}} = \frac{168000000}{(0 + 1) \times (19 + 1) \times 32} = 262500$$

在实际应用中，可以根据工程里的正弦波点数和定时器配置生成特定频率的正弦波。

DMA 配置

本工程的数据传输由 DMA 完成，其代码见代码清单 39-7。

代码清单 39-7 DMA 配置

```

1
2  /* 波形数据 ----- */
3  const uint16_t Sine12bit[32] = {
4      2048 , 2460 , 2856 , 3218 , 3532 , 3786 , 3969 , 4072 ,
5      4093 , 4031 , 3887 , 3668 , 3382 , 3042 , 2661 , 2255 ,
6      1841 , 1435 , 1054 , 714 , 428 , 209 , 65 , 3 ,
7      24 , 127 , 310 , 564 , 878 , 1240 , 1636 , 2048
8
9  };
10
11  uint32_t DualSine12bit[32];
12
13  /**

```

```
14  * @brief   DAC 初始化函数
15  * @param   无
16  * @retval  无
17  */
18 void DAC_Mode_Init(void)
19 {
20     uint32_t Idx = 0;
21
22     DAC_Config();
23     DAC_TIM_Config();
24
25     /* 填充正弦波形数据，双通道右对齐 */
26     for (Idx = 0; Idx < 32; Idx++) {
27         DualSine12bit[Idx] = (Sine12bit[Idx] << 16) + (Sine12bit[Idx]);
28     }
29
30     DAC_DMA_Config();
31 }
32
33 /**
34  * @brief   配置 DMA
35  * @param   无
36  * @retval  无
37  */
38 static void DAC_DMA_Config(void)
39 {
40     DMA_InitTypeDef  DMA_InitStructure;
41
42     /* DAC1 使用 DMA1 通道 7 数据流 5 时钟 */
43     RCC_AHB1PeriphClockCmd(DAC_DMA_CLK, ENABLE);
44
45     /* 配置 DMA2 */
46     DMA_InitStructure.DMA_Channel = DAC_CHANNEL;
47     DMA_InitStructure.DMA_PeripheralBaseAddr = DAC_DHR12RD_Address; //外设数据地址
48     //内存数据地址 DualSine12bit
49     DMA_InitStructure.DMA_Memory0BaseAddr = (uint32_t)&DualSine12bit;
50     DMA_InitStructure.DMA_DIR = DMA_DIR_MemoryToPeripheral; //数据传输方向内存至外设
51     DMA_InitStructure.DMA_BufferSize = 32; //缓存大小为 32 字节
52     DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable; //外设数据地址固定
53     DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable; //内存数据地址自增
54     //外设数据以字为单位
55     DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
56     DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Word; //内存数据以字为单位
57     DMA_InitStructure.DMA_Mode = DMA_Mode_Circular; //循环模式
58     DMA_InitStructure.DMA_Priority = DMA_Priority_High; //高 DMA 通道优先级
59     DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
60     DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;
61     DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;
62     DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
63
64     DMA_Init(DAC_DMA_STREAM, &DMA_InitStructure);
65
66     /* 使能 DMA_Stream */
67     DMA_Cmd(DAC_DMA_STREAM, ENABLE);
68 }
```

在上述代码中，定义了由脚本得到的正弦波数据表 Sine12bit 变量，一共为 32 个点。在 DAC_Mode_Init 函数中，调用了前面介绍的 DAC_Config 和 DAC_TIM_Config 初始化 DAC 和定时器，然后在 for 循环中把单通道的正弦波数据表 Sine12bit 复制扩展成为双通道的数据 DualSine12bit，扩展后的数据将会直接被 DMA 搬运至 DAC 的 DHR12RD 寄存器中。

复制完数据后，DAC_Mode_Init 调用下面的 DAC_DMA_Config 函数初始化 DMA，配置的重点是要设置好 DHR12RD 寄存器的地址，正弦波数据的内存地址（注意是双通道数据 DualSine12bit），DMA 缓存的个数（即单个周期的正弦波点数）以及 DMA 工作在循环模式。

经过这样的配置后，定时器每间隔一定的时间就会触发 DMA 搬运双通道正弦波表的一个数据到 DAC 双通道寄存器进行转换，每完成一个周期后 DMA 重新开始循环，从而达到连续输出波形的目的。

主函数

代码清单 39-8 主函数

```
1 /**
2  * @brief 主函数
3  * @param 无
4  * @retval 无
5  */
6 int main(void)
7 {
8
9     LED_GPIO_Config();
10
11     /*初始化串口*/
12     Debug_USART_Config();
13
14     printf("\r\n 欢迎使用野火 STM32 F407 开发板.\r\n");
15     printf("\r\n DAC 输出例程, 输出正弦波\r\n");
16     printf("\r\n 使用示波器检测开发板的 PA4、PA5 引脚, 可测得正弦波\r\n ");
17
18     /*初始化 DAC, 开始 DAC 转换, 使用示波器检测 PA4/PA5, 可观察到正弦波*/
19     DAC_Mode_Init();
20
21     while (1);
22 }
```

本工程的主函数非常简单，直接调用 DAC_Mode_Init 即可完成所有的配置，此时再使用示波器测量 PA4、PA5 引脚可查看其输出的波形。。

39.4.3 下载验证

注意：参考本章的硬件设计部分说明，实验前需要拆掉 SPI-FLASH 芯片附近的电阻 R6。

用 USB 线连接开发板的“USB 转串口”接口跟电脑，把编译好的程序下载到开发板，使用示波器测量 PA4、PA5 的引脚可看到正弦波形（示波器使用 x10 倍档测量更加准确），见图 39-8，注意观察图中示波器测量出的频率值和电压峰值。

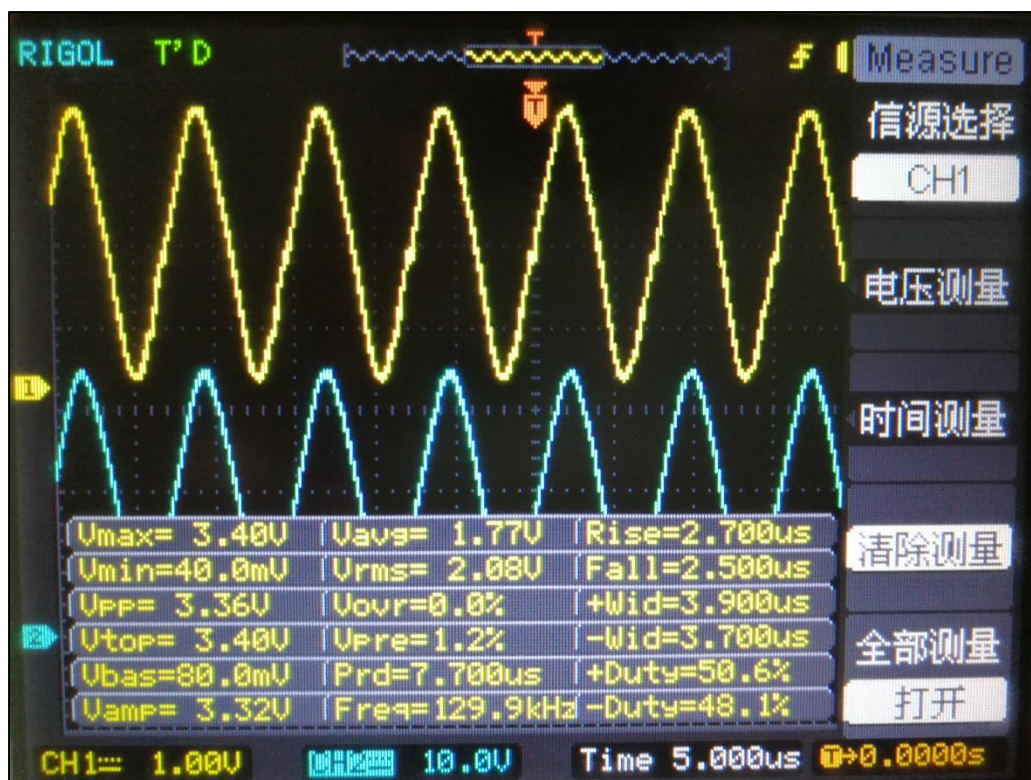


图 39-8 使用示波器测量出的电压波形